Téma 6: Úvod do OOP (Objektově orientované programování)

Teoretický základ

OOP spojuje data a kód, který nad nimi pracuje, do jednoho celku. Místo samostatných proměnných tvoříme objekty simulující reálný svět.

- **Třída (Class):** Výrobní šablona, návrh. Pojmenováváme s velkým písmenem (např. `class Auto:`).
- **Objekt (Instance):** Konkrétní produkt vytvořený podle třídy (např. moje červená škodovka v garáži).
- **Konstruktor (`__init__`):** Speciální metoda, která se volá při zrodu objektu a nastavuje jeho vlastnosti (atributy).
- **`self`:** Odkaz objektu na sebe sama. Musí být vždy prvním parametrem každé metody uvnitř třídy, aby instance věděla, s čí pamětí pracuje.

Základní koncept k zamyšlení: Dědičnost (Inheritance)

Pokud máme naprogramovanou obecnou třídu `Zviratko`, která má definovanou schopnost (metodu) `dychej()`, a poté vytvoříme podtřídu `Pes`, která od ní *dědí*, pes automaticky získá metodu `dychej()`. Může navíc přidat vlastní unikátní metody, např. `zastekej()`. Dědičnost zabraňuje psaní duplicitního kódu.

Úkol 1 (Lehká obtížnost)

Zadání úkolu: Vytvořte třídu (šablonu) pojmenovanou `Pes`. Třída bude mít povinný konstruktor, který přijme jméno. Bude obsahovat také metodu `zastekej()`, která vypíše *"Haf! Jsem pes [Jméno]."* Vytvořte jeden objekt psa a zavolejte jeho štěkání.

Řešení úkolu 1

```
class Pes:
    def   init  (self, jmeno):
        # self.jmeno ukládá údaj bezpečně pro konkrétní novou instanci
        self.jmeno = jmeno

    def zastekej(self):
        # Metoda zastekej k údaji přistupuje přes identifikátor self
        print(f"Haf! Jsem pes {self.jmeno}.")

# "Zrození" objektu do paměti a zavolání metody
moje stene = Pes("Rex")
moje_stene.zastekej()
```

Úkol 2 (Střední obtížnost)

Zadání úkolu: Rozšiřte předchozí příklad s Psem. Kromě jména přidejte do konstruktoru atribut `hlad` napevno nastavený na hodnotu `True`. Vytvořte metodu `nakrm()`, která tento atribut přepne na hodnotu `False` a vypíše poděkování. Pokusí-li se jíst najezený pes, vypište upozornění.

Řešení úkolu 2

```
class Pes:
    def   init  (self, jmeno):
        self.jmeno = jmeno
        self.hlad = True  # Výchozí vnitřní stav zvířete

    def nakrm(self):
        # Měníme stav objektu na základě jeho aktuálních hodnot
        if self.hlad == True:
            self.hlad = False
            print(f"{self.jmeno} spokojeně zhltnul večeři.")
        else:
            print(f"{self.jmeno} už nemá hlad a jídlo ignoruje.")

pes = Pes("Barvk")
pes.nakrm() # Barvk jí
pes.nakrm() # Nyní odmítne
```

Úkol 3 (Těžká obtížnost)

Zadání úkolu: Vytvořte bázovou třídu `Zamestnanec` s atributy jméno a plat, a metodu `vypocti_mzdu()`, která plat vrátí. Poté vytvořte potomka `Manazer` dědícího od Zaměstnance. Ten v konstruktoru přidá `bonus` a přepíše (overridne) metodu pro výpočet mzdy tak, aby vracela součet platu a bonusu.

Řešení úkolu 3

```
class Zamestnanec:
    def   init  (self, jmeno, zakladni_plat):
        self.jmeno = jmeno
        self.zakladni_plat = zakladni_plat

    def vypocti_mzdu(self):
        return self.zakladni_plat

# Manažer DĚDÍ vlastnosti Zaměstnance
class Manazer(Zamestnanec):
    def   init  (self, jmeno, zakladni_plat, bonus):
        # super() převezme inicializaci z nadřazené bázové třídy
        super().  init  (jmeno, zakladni_plat)
        self.bonus = bonus

    # Polymorfismus - nahrazení chování metody podle kontextu
    def vypocti_mzdu(self):
        return self.zakladni_plat + self.bonus

b = Zamestnanec("Karel", 30000)
m = Manazer("Šéf", 50000, 20000)
print(f"Mzda dělníka: {b.vypocti_mzdu()}, Mzda manažera:
{m.vypocti_mzdu()}")
```

Úkol 4 (Lehká obtížnost - dodatek)

Zadání úkolu: Vytvořte jednoduchou třídu `Kniha`. Bude mít v konstruktoru (`__init__`) atributy `nazev` a `autor`. Přidejte jí metodu `info()`, která při zavolání úhledně oba údaje spojí a vypíše na konzoli.

Řešení úkolu 4

```
class Kniha:
    def   init  (self, nazev, autor):
        self.nazev = nazev
        self.autor = autor

    def info(self):
        # Nezapomeňte u přístupu k atributům VŽDY používat předponu self.
        print(f"Kniha: {self.nazev} (od {self.autor})")

# Vytvoření instance (fyzického objektu) a vyvolání informací
moje_kniha = Kniha("RUR", "Karel Čapek")
moje_kniha.info()
```

Úkol 5 (Střední obtížnost - dodatek)

Zadání úkolu: Napište třídu `BankovniUcet`, která v konstruktoru nastaví `zustatek` napevno na 0. Vytvořte metody `vloz(castka)` a `vyber(castka)`. Pokud se uživatel pokusí vybrat více, než je na účtu, nesmí se mu zapsat záporná bilance a metoda musí vypsat chybovou hlášku!

Řešení úkolu 5

```
class BankovniUcet:
    def   init  (self):
        self.zustatek = 0

    def vloz(self, castka):
        self.zustatek += castka

    def vyber(self, castka):
        if castka > self.zustatek:
            print("Chyba: Nedostatečný zůstatek pro transakci!")
        else:
            self.zustatek -= castka
            print(f"Vybráno {castka}. Zůstatek: {self.zustatek}")

ucet = BankovniUcet()
ucet.vloz(1000)
ucet.vyber(1500) # Spustí ochranu proti přečerpání
```

Úkol 6 (Těžká obtížnost - dodatek)

Zadání úkolu: (Polymorfismus). Vytvořte bázovou prázdnou třídu `Tvar`, která bude definovat nicnedělající metodu `obsah()`. Od ní nechte zdědit dvě podtřídy: `Ctverec` (v konstruktoru saje parametr strana) a `Obdelnik` (parametry a, b). V obou podtřídách přepište `obsah()` pro správný matematický výpočet.

Řešení úkolu 6

```
class Tvar:
    def obsah(self):
        pass # Prázdná obecná šablona pro potomky

class Ctverec(Tvar):
    def   init  (self, strana):
        self.strana = strana
    def obsah(self):
        return self.strana ** 2   # Přepis chování rodiče

class Obdelnik(Tvar):
    def   init  (self, a, b):
        self.a = a
        self.b = b
    def obsah(self):
        return self.a * self.b    # Zcela jiný přepis chování rodiče

t1 = Ctverec(5)
t2 = Obdelnik(4, 5)
print(f"Čtverec: {t1.obsah()}, Obdélník: {t2.obsah()}")
```